



LocalStorage, SessionStorage y Timers en JavaScript

Aprende a gestionar datos en el navegador y controlar la ejecución temporal de tu código JavaScript



¿Qué es LocalStorage y SessionStorage?

Son como **cajones digitales donde guardamos información directamente en el navegador** del usuario. Ambos nos permiten almacenar datos localmente sin necesidad de una base de datos externa.

LocalStorage

La información permanece guardada incluso después de cerrar el navegador. Los datos persisten hasta que el usuario los elimine manualmente o uses código para borrarlos.

SessionStorage

La información se borra automáticamente al cerrar la pestaña o ventana del navegador. Ideal para datos temporales de una sesión específica.

Métodos Fundamentales de Storage

Ambos tipos de storage comparten los mismos métodos para manipular datos. Dominar estos cuatro métodos te permitirá gestionar completamente la información almacenada.

1 `setItem(clave, valor)`

Guarda un nuevo dato o actualiza uno existente. La clave actúa como identificador único para recuperar el valor posteriormente.

3 `removeItem(clave)`

Elimina un elemento específico del storage usando su clave identificadora.

2 `getItem(clave)`

Recupera el valor asociado a una clave específica. Devuelve null si la clave no existe en el storage.

4 `clear()`

Borra completamente todos los datos almacenados en el storage correspondiente.

Ejemplo Práctico con LocalStorage

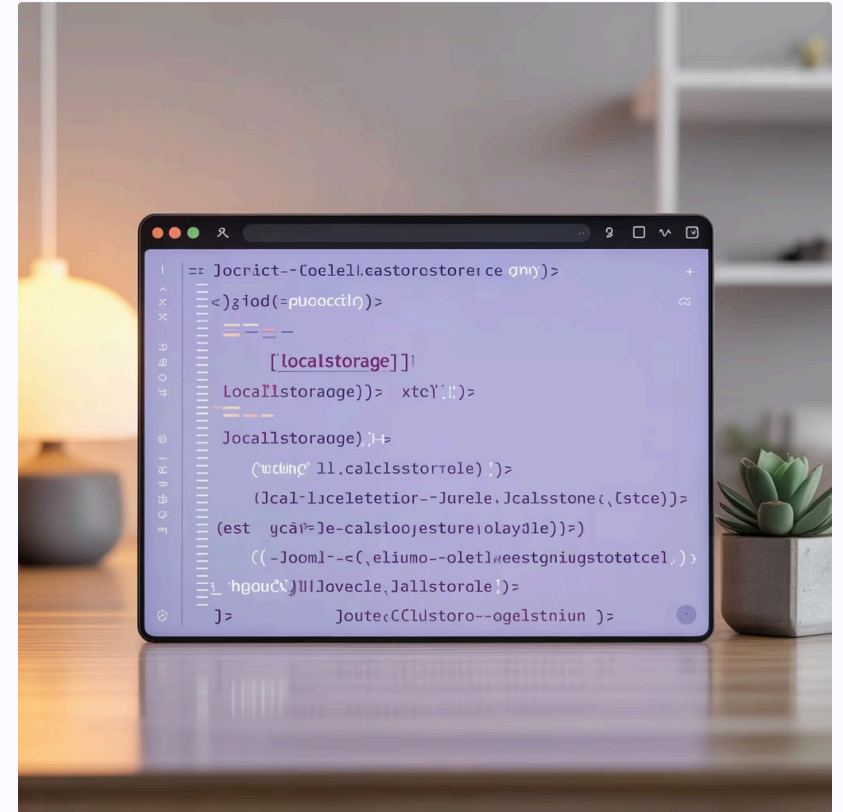
Código de ejemplo:

```
// Guardar información
localStorage.setItem("nombre", "Carlos");
localStorage.setItem("edad", "25");

// Leer información
console.log(localStorage.getItem("nombre"));
// Resultado: "Carlos"

// Borrar un elemento específico
localStorage.removeItem("edad");

// Borrar todo el contenido
localStorage.clear();
```



Este ejemplo muestra cómo almacenar, recuperar y eliminar datos de forma sencilla.



```
Name : <Example">  
Value : 123>
```

Trabajando con Objetos: JSON.stringify y JSON.parse

Problema: LocalStorage y SessionStorage solo pueden almacenar cadenas de texto (strings). ¿Cómo guardamos objetos complejos o arrays?

1

Objeto JavaScript

Datos estructurados que queremos almacenar

```
const usuario = {  
  nombre: "Ana",  
  edad: 20,  
  email: "ana@email.com"  
};
```

2

JSON.stringify

Convierte el objeto en una cadena de texto

```
localStorage.setItem("usuario",  
  JSON.stringify(usuario));
```

3

JSON.parse

Convierte la cadena de vuelta a objeto

```
const data = JSON.parse(  
  localStorage.getItem("usuario"));  
console.log(data.nombre); // "Ana"
```

SessionStorage en Acción

Ejemplo práctico:

```
// Guardar preferencia temporal
sessionStorage.setItem("color", "azul");
sessionStorage.setItem("idioma", "español");

// Leer preferencia
const colorActual = sessionStorage.getItem("color");
console.log(colorActual); // "azul"

// Al cerrar la pestaña, todos los datos
// de sessionStorage desaparecen automáticamente
```

Diferencia clave: SessionStorage es perfecto para datos que solo necesitas durante una sesión específica, como preferencias temporales o estados de formularios.

Introducción a los Timers

Los timers nos permiten **controlar cuándo se ejecuta nuestro código**, añadiendo dimensión temporal a nuestras aplicaciones. Son fundamentales para animaciones, notificaciones y funcionalidades programadas.



setTimeout

Ejecuta código **una sola vez** después de un tiempo determinado. Perfecto para retrasos programados.



setInterval

Ejecuta código **repetidamente** cada cierto intervalo de tiempo. Ideal para actualizaciones periódicas.



clearInterval

Detiene la ejecución de un setInterval activo. Esencial para controlar cuándo parar las repeticiones.

setTimeout: Ejecutar Código con Retraso

Código de ejemplo:

```
console.log("Inicio");

setTimeout(() => {
  console.log("Han pasado 2 segundos");
}, 2000);
```

Salida en consola:

```
Inicio
Han pasado 2 segundos
```



Concepto clave: JavaScript no se detiene esperando al setTimeout. El código continúa ejecutándose inmediatamente, y la función del setTimeout se ejecuta después del tiempo especificado.

Esto se llama **ejecución asíncrona** y es fundamental en JavaScript moderno.

setInterval y clearInterval: Repetición Controlada

```
let contador = 0;

const intervalo = setInterval(() => {
  contador++;
  console.log("Contador:", contador);

  if (contador === 5) {
    clearInterval(intervalo);
    console.log("Intervalo detenido");
  }
}, 1000);
```

¿Qué hace este código?

- Incrementa un contador cada segundo (1000 milisegundos)
- Muestra el valor actual del contador en consola
- Cuando llega a 5, detiene automáticamente la repetición
- Es importante usar `clearInterval` para evitar que el timer siga ejecutándose indefinidamente



Resumen: Tu Kit de Herramientas Completo



Storage Persistente

LocalStorage para datos permanentes que sobreviven al cierre del navegador. Perfecto para configuraciones de usuario y preferencias.



Storage Temporal

SessionStorage para datos que solo necesitas durante la sesión actual. Ideal para carritos de compra temporales.



Conversión de Datos

JSON.stringify/parse para convertir objetos complejos en strings almacenables y viceversa.



Control Temporal

setTimeout/setInterval para ejecutar código en momentos específicos o de forma repetitiva, con **clearInterval** para control total.

Con estas herramientas ya puedes crear aplicaciones web más dinámicas y con mejor experiencia de usuario. ¡A practicar!

